

LINKED LIST AND TREES

DSA course notes - from pointers and nodes to tree traversal

Core idea

A linked list is a chain of connected nodes. A tree uses the same node-and-pointer idea, but allows branching.

What changes?

Linked List: one next link. Binary Tree: left and right links.

Why this order?

Pointers and traversal become familiar in linked lists before the same ideas are used recursively in trees.

Topics covered

Node structure, head pointer, traversal, insertion, deletion, reversal, doubly linked list, recursion, binary tree construction, and preorder/inorder/postorder traversal.

1. LINKED LIST

A linked list is made of nodes. Each node stores a value and a pointer to another node. The nodes do not need to sit next to each other in memory; the pointers connect them.

10 -> 20 -> 30 -> NULL

Node structure

```
struct Node {
    int data;
    Node* next;
};
```

data stores the value. next stores the address of the next node.

Creating a node

```
struct Node {
    int data;
    Node* next;

    Node(int x) {
        data = x;
        next = nullptr;
    }
};
```

```
Node* a = new Node(10);
```

After the allocation, a points to a node containing 10 and whose next pointer is nullptr.

2. HEAD AND TRAVERSAL

The list needs a pointer to its first node. We usually call it head.

```
head
|
v
10 -> 20 -> 30 -> nullptr
```

If head == nullptr, the list is empty. Head is a pointer; it is not itself a node.

Traversal means starting at head and moving through the next pointers until nullptr.

TRAVERSAL

```
Node* temp = head;

while (temp != nullptr) {
    cout << temp->data << " ";
    temp = temp->next;
}
```

The line temp = temp->next is the whole movement of a singly linked list: follow the link stored in the current node.

3. INSERTION AND DELETION

At the beginning, insertion is constant time because we only change two pointers.

Insert at beginning

```
Node* x = new Node(10);
x->next = head;
```

```
head = x;
```

Before: 20 -> 30 -> 40 -> nullptr

After: 10 -> 20 -> 30 -> 40 -> nullptr

Complexity: $O(1)$.

Insert at end

```
Node* temp = head;
while (temp->next != nullptr)
    temp = temp->next;
```

```
temp->next = new Node(40);
```

Without a tail pointer, we walk to the last node, so the complexity is $O(N)$.

Delete first node

```
Node* temp = head;
head = head->next;
delete temp;
```

Complexity: $O(1)$.

4. REVERSE A LINKED LIST

Reversal is a pointer problem. Do not move values around; change the direction of the links.

10 -> 20 -> 30 -> 40 -> nullptr

40 -> 30 -> 20 -> 10 -> nullptr

Use three pointers: prev, cur, and next. Save the old next node before breaking the link.

CORE CODE

```
Node* prev = nullptr;
Node* cur = head;

while (cur != nullptr) {
    Node* next = cur->next;
    cur->next = prev;
    prev = cur;
    cur = next;
}
```

```
head = prev;
```

One iteration changes this:

prev	cur
v	v
10 ->	20 -> 30

into:

```
nullptr <- 10    20 -> 30
```

The list is reversed in $O(N)$ time and $O(1)$ extra space.

5. DOUBLY LINKED LIST

A doubly linked list stores both the previous node and the next node.

```
nullptr <- 10 <-> 20 <-> 30 -> nullptr
```

Node structure

```
struct Node {  
    int data;  
    Node* prev;  
    Node* next;  
};
```

The extra pointer lets us move in both directions. The important idea is still the same: nodes are connected through pointers.

6. RECURSION WITH A LINKED LIST

A linked list naturally supports recursion because every node leads to one next node.

```
void print(Node* node) {  
    if (node == nullptr)  
        return;  
  
    cout << node->data << " ";  
    print(node->next);  
}  
  
print(10)  
|  
+--> print(20)  
    |  
    +--> print(30)  
        |  
        +--> print(nullptr)
```

The recursive call simply follows the next pointer. This is the bridge to trees.

7. FROM LINKED LIST TO TREE

The node-and-pointer idea stays the same. The structure changes because a node can now point to more than one child.

Linked List	Binary Tree
data next	data left right
A -> B -> C -> D	<pre> A / \ B C</pre>

A linked list has one obvious next node. A binary tree can have a left child and a right child. That branching is what makes tree traversal different.

Binary tree node

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
  
    Node(int x) {  
        data = x;  
        left = right = nullptr;  
    }  
};
```

```
    }
};
```

8. BUILDING A BINARY TREE

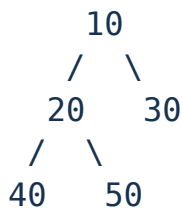
Create the root, then connect child nodes using left and right pointers.

EXAMPLE

```
Node* root = new Node(10);

root->left = new Node(20);
root->right = new Node(30);

root->left->left = new Node(40);
root->left->right = new Node(50);
```



Compare the pointer assignments:

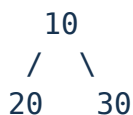
Linked List:
a->next = b;

Binary Tree:
a->left = b;
a->right = c;

The syntax is almost the same. The tree simply has two directions instead of one.

9. WHY TREE TRAVERSAL USES RECURSION

In a linked list, from one node there is one next node. In a binary tree, from one node there can be two child subtrees.

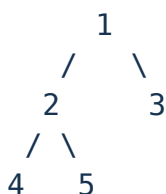


A recursive DFS therefore follows both pointers:

```
solve(root->left);
solve(root->right);
```

The base case is the same idea as a linked list: stop when the pointer is null.

10. TREE TRAVERSALS



PREORDER - Root, Left, Right

```
void preorder(Node* root) {
    if (root == nullptr)
        return;
```

```

    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

```

Output: 1 2 4 5 3

INORDER - Left, Root, Right

```

void inorder(Node* root) {
    if (root == nullptr)
        return;

    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

```

Output: 4 2 5 1 3

POSTORDER - Left, Right, Root

```

void postorder(Node* root) {
    if (root == nullptr)
        return;

    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}

```

Output: 4 5 2 3 1

11. THE CONNECTION TO REMEMBER

Linked List	Binary Tree
Node stores data + next pointer.	Node stores data + left/right pointers.
One path to follow.	Branching paths to follow.
Traversal: follow next.	Traversal: explore left and right.
Recursion follows one next pointer.	Recursion can explore two subtrees.

The most useful mental model is simple: a tree does not introduce a new pointer idea. It takes the same node-and-pointer model from linked lists and adds branching.

Before trees, make sure these linked-list ideas are comfortable: creating nodes, using pointers, following links, handling nullptr, and writing recursive traversal. Once those are clear, the basic binary-tree code is much easier to read.

QUICK REFERENCE

Linked List node->next	Binary Tree node->left / node->right
Move: temp = temp->next	Explore: dfs(root->left), dfs(root->right)
End: temp == nullptr	Leaf: left == nullptr && right == nullptr